

Document number: P0412R0
Date: 2016-07-05
Project: EWG, LEWG
Reply-to: Mikhail Maltsev <maltsevm@gmail.com>

Benchmarking Primitives

1. [Motivation](#)
2. [Design space](#)
3. [Proposal](#)
4. [Naming](#)
5. [Implementation experience](#)
6. [Example](#)
7. [Acknowledgements](#)
8. [References](#)

1. Motivation

When optimizing program performance, it is often desirable to be able to measure performance of an isolated piece of code on some predefined input. Ideally such measurement:

- Should not be affected by I/O timing
- Should be done on optimized code
- Should not be affected by compiler optimizations based on the fact that input data is known in advance

For example, consider the following code:

```
#include <chrono>
#include <iostream>

double perform_computation(int);

void benchmark()
{
    using namespace std;
    auto start = chrono::high_resolution_clock::now();
    double answer = perform_computation(42);
    auto delta = chrono::high_resolution_clock::now() - start;
    std::cout << "The answer is: " << answer << ". Computation took "
              << chrono::duration_cast(delta).count()
              << " ms";
}
```

Suppose that `perform_computation` does not have any observable side effect. In this case, the compiler can perform constant-folding, i.e., compute the value of `answer` at compile time. It can also move computation before the first call to `now` or after the second one.

It would be nice to have a portable way to disable such optimizations.

2. Design space

Proposal P0342R0 [1] to add timing barriers was rejected at Oulu meeting.

Chandler: If the timing fence is inside `now`, and `now` is in another TU, how does the compiler know there is a fence?

...

Chandler: I was sympathetic to start, but I don't think I can implement this. The only way I can implement this is to undo as-if. I have to make sure everything can not move across the fence.

Hal: I agree — I don't think we can implement

Chandler also mentioned his talk [2] at CppCon 2015. This talk describes two primitives that can be used for benchmarking (using GCC extended asm syntax):

```
static void escape(void* p)
```

```
{
    asm volatile("" : : "g"(p) : "memory");
}

static void clobber()
{
    asm volatile("" : : : "memory");
}
```

IMHO, standardizing these functions (especially `clobber`) is problematic because the semantics of a "memory" clobber in the clobber list of an inline asm is implementation-specific.

3. Proposal

This proposal adds a new header `<benchmark>`, which defines the following function templates:

```
namespace std {
namespace experimental {
namespace benchmark {

template<class T>
void keep(T &&) noexcept;

template<class T>
void touch(T &) noexcept;

} } }
```

The implementation shall treat a call to `keep` as-if `keep` outputs each byte of its argument's object representation into an unspecified output device.

The implementation shall treat a call to `touch` as-if `touch` reads each byte of its argument's object representation from an unspecified input device. The actual value of the argument remains unchanged, but the implementation is not allowed to rely on that when performing optimization. If `T` is const-qualified, the program is ill-formed.

Note: implementations are encouraged to leverage the as-if principle and not perform any real I/O.

4. Naming

Alternative names for the `keep` function:

- `do_not_optimize` — used in Google benchmark [3] library
- `do_not_optimize_away` — used in Celero [4] library
- `escape`

Alternative names for the `touch` function:

- `clobber`

Alternative names for the header and the namespace:

- `bench`
- `benchmark`
- `benchmarking`

5. Implementation experience

This proposal has been implemented using GCC/Clang-specific language extensions (namely, extended inline assembly). It is also possible to implement `keep` and `touch` functions for POD types that don't have padding. Both prototype implementations are available on Github at <https://github.com/miyuki-chan/benchmarking-proposal>. Implementation for the general case is likely to require compiler support (i.e., built-in a.k.a. *intrinsic* functions).

6. Example

The code shown above could be rewritten as:

```
#include <chrono>
#include <iostream>
#include <benchmark>
```

```
double perform_computation(int);

void benchmark()
{
    using namespace std;
    auto start = chrono::high_resolution_clock::now();
    int value = 42;
    experimental::benchmark::touch(value);
    double answer = perform_computation(value);
    experimental::benchmark::keep(answer);
    auto delta = chrono::high_resolution_clock::now() - start;
    std::cout << "The answer is: " << answer << ". Computation took "
              << chrono::duration_cast<chrono::milliseconds>(delta).count()
              << " ms";
}
```

This avoids the mentioned problems with constant folding and code motion.

7. Acknowledgements

Thanks to Walter Brown for useful feedback.

8. References

1. Mike Spertus, Timing barriers, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0342r0.html>
2. Chandler Carruth, Tuning C++: Benchmarks, and CPUs, and Compilers! Oh My! <https://www.youtube.com/watch?v=nXaxk27zwIk>
3. Google benchmark library, <https://github.com/google/benchmark>
4. Celero library, <https://github.com/DigitalInBlue/Celero>